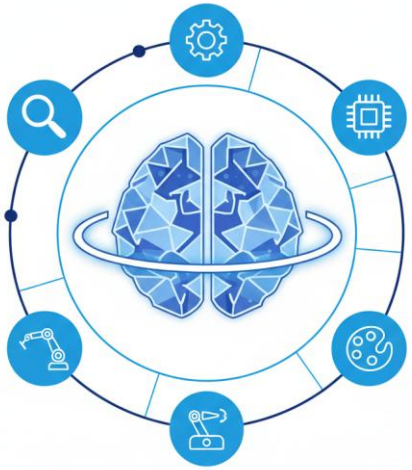
AI 필수 코딩 언어 – 파이썬

python 기초 문법



AI 시대의 필수 코딩 언어

- **파이썬(Python)**

+ 파이썬에 대해 알려 줘

🐍 파이썬이 뭐야?

- **1991년** 귀도 반 로섬(Guido van Rossum)이 만든 언어
- 문법이 **사람 말처럼 직관적**
- **인터프리터 언어** → 코드 작성 후 바로 실행 가능

🔧 파이썬으로 뭘 할 수 있나?

- 📊 **데이터 분석 & 시각화**
- 🤖 **인공지능 / 머신러닝**
- 🌐 **웹 서버 & API 개발**
- 🧪 **자동화 스크립트**
- 🎮 **간단한 게임**
- 📁 **파일 처리, 크롤링**

AI 시대의 필수 코딩 언어

- **파이썬 설치 유무 확인**

명령 프롬프트

```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LG>python --version
Python 3.14.2

C:\Users\LG>where python
C:\Users\LG\AppData\Local\Programs\Python\Python314\python.exe
C:\Users\LG\AppData\Local\Microsoft\WindowsApps\python.exe

C:\Users\LG>
```

··· 사용자 > LG > AppData > Local > Programs > Python >

정렬 보기 ···

이름	수정한 날짜	유형	크기
Launcher	2026-01-08 오전 7:13	파일 폴더	
Python314	2026-01-19 오후 4:49	파일 폴더	

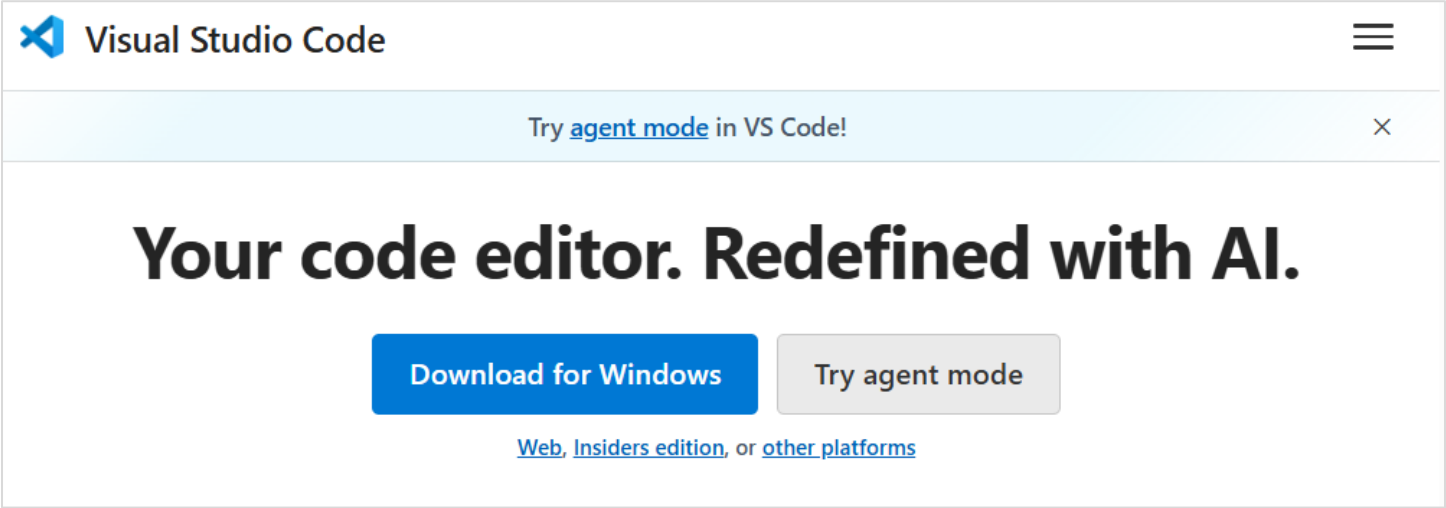
AI 시대의 필수 코딩 언어

- 파이썬 - www.python.org > Downloads

체크 후 Install Now
어떤 경로(PATH)에서든
실행될수 있음

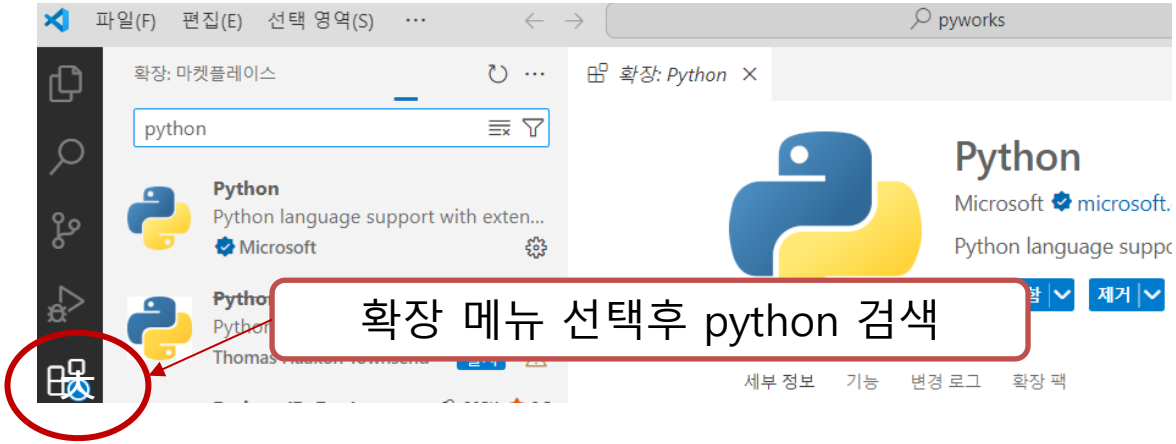
AI 시대의 필수 코딩 IDE

◆ VS code 설치

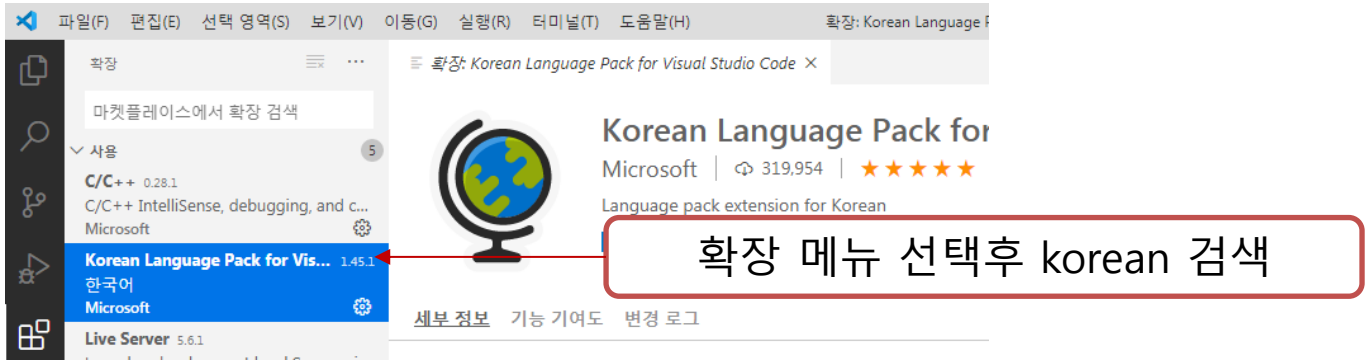


Visual Studeio Code

◆ Python 언어 지원 확장 팩 설치하기

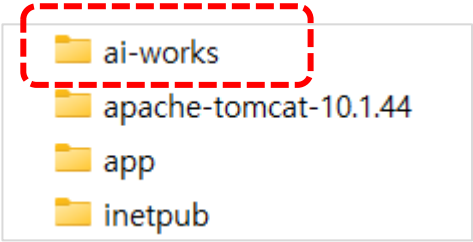


◆ 한국어 팩 설치

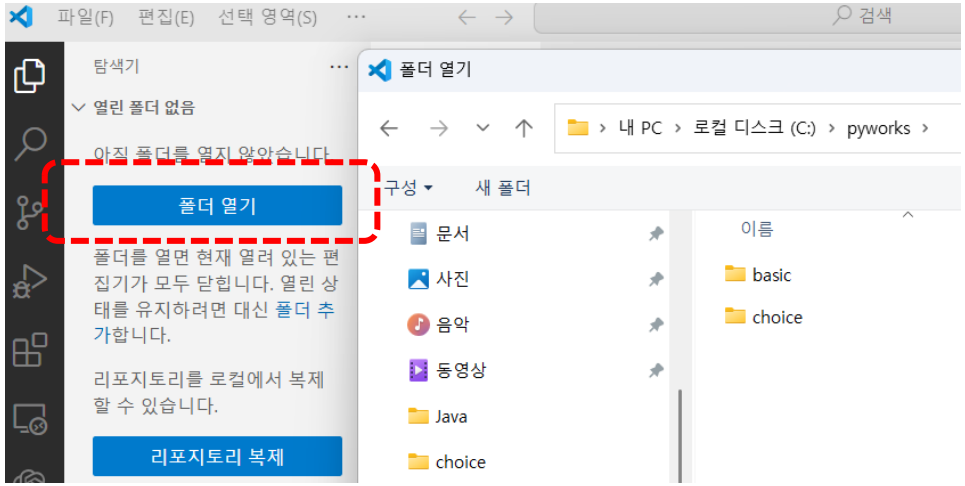


Visual Studeio Code

◆ VS code – 작업 폴더 설정



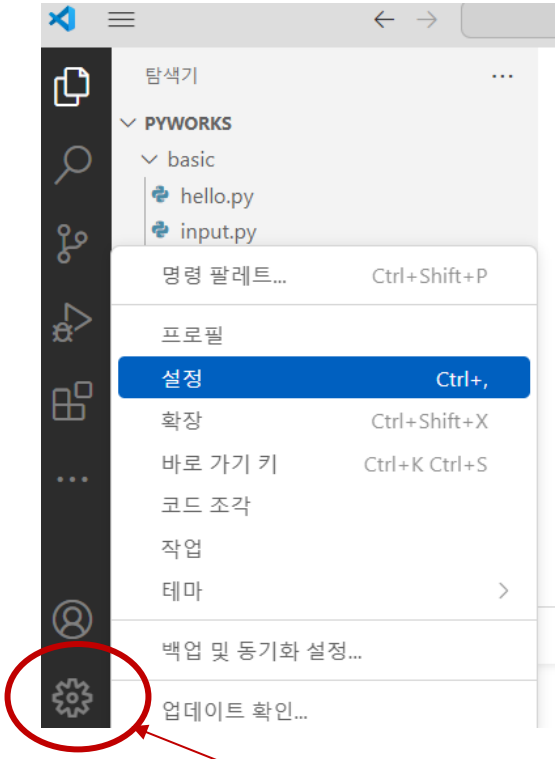
① 작업영역 폴더 만들기



② 작업영역 폴더 설정

Visual Studeio Code

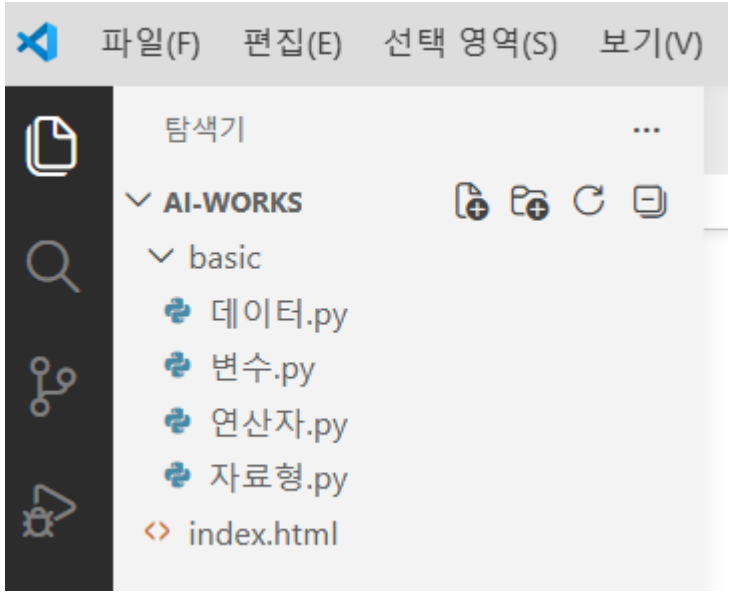
◆ VS code – 관리 도구



관리 > 설정

Visual Studeio Code

- **디렉터리(폴더) 구조**



출력 – print()

- **데이터 출력 – print() 함수**

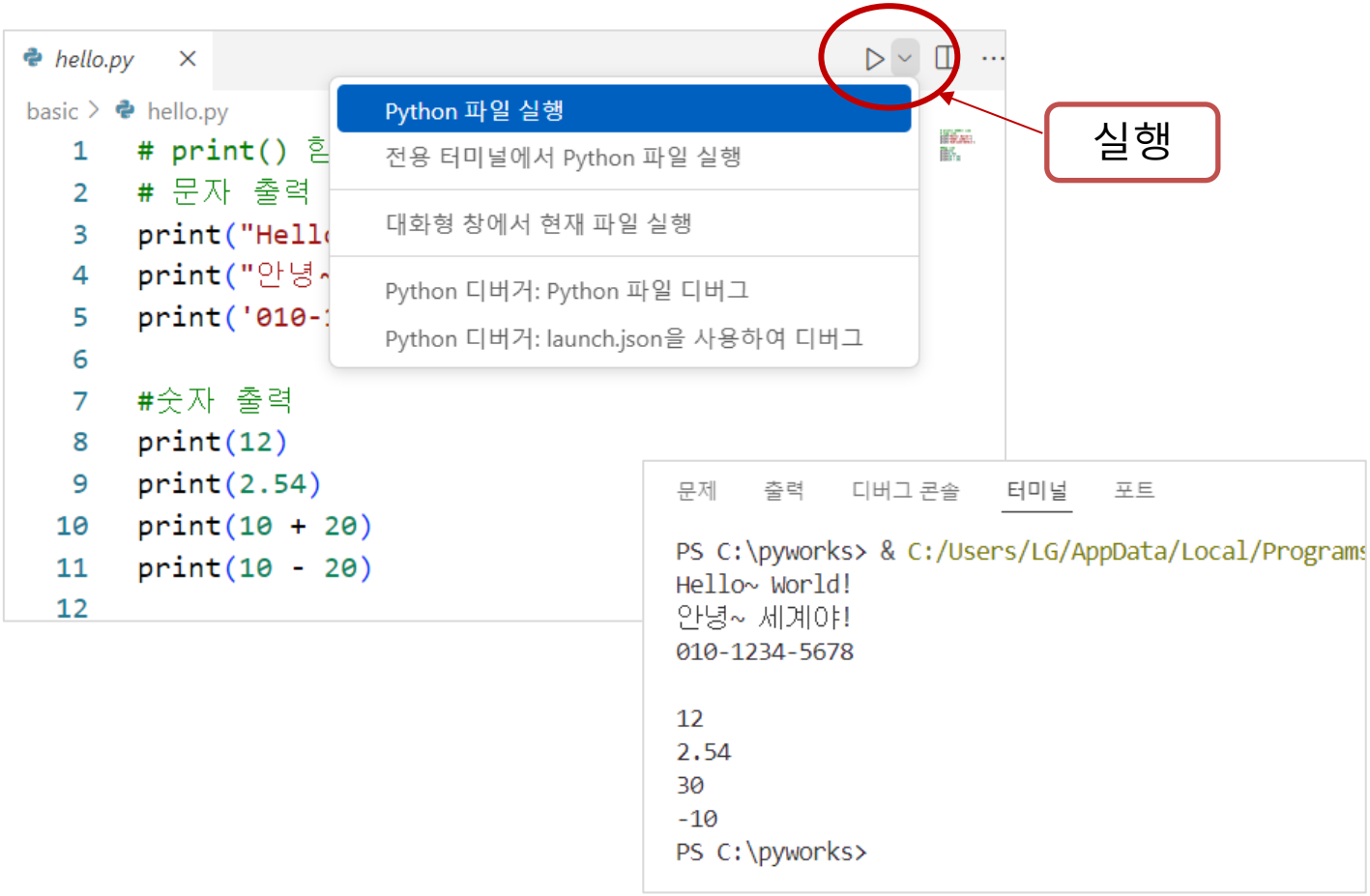
```
# 출력 - print() 함수는 괄호 안의 내용을 출력하는 함수.
# 문자 - 따옴표로 감싸진 글자
print('A')
print("coffee")
print("아메리카노")
# print(커피) # 오류 발생, 문자열이 아니고 변수로 인식됨

# 숫자 - 따옴표로 감싸지 않은 글자, 정수, 실수
print(123)
print(3.14)

# 연산 - 숫자 사이에 사칙연산 기호를 사용하여 계산.
print(1 + 2)  # 덧셈
print(5 - 3)  # 뺄셈
print(4 * 2)  # 곱셈
print(10 / 2) # 나눗셈
```

```
A
coffee
아메리카노
123
3.14
3
2
8
5.0
```

#을 붙이면 한 줄(행)이 주석이 되어 기계어 코드로 번역되지 않음
단축키 : ctrl + '/'

출력 – print()

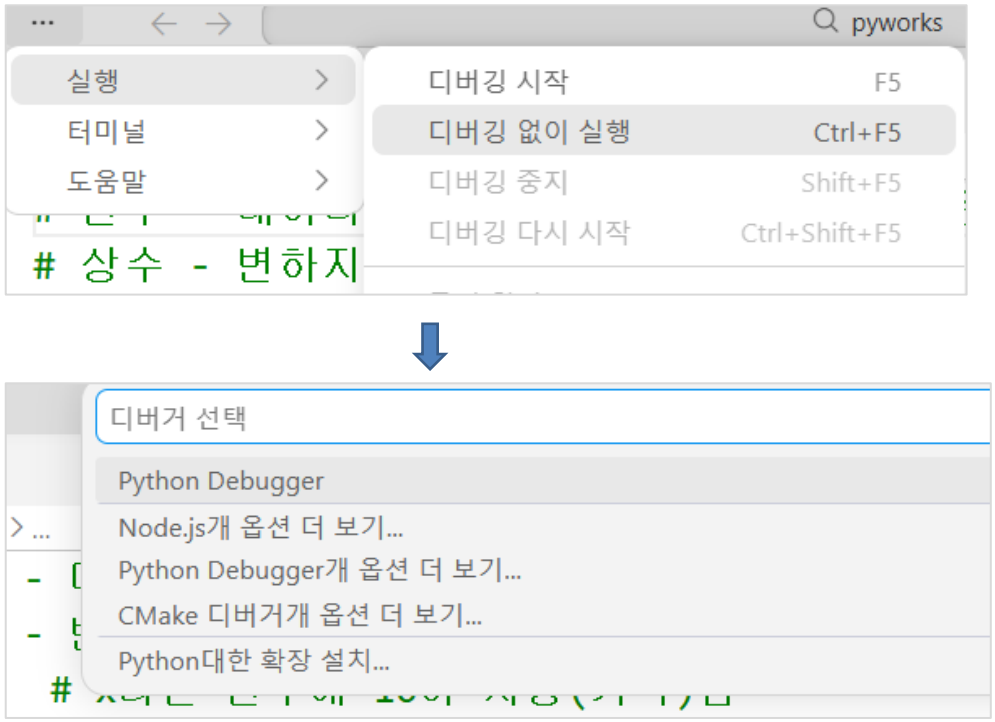
- **파일 실행 및 터미널 출력 1**



출력 – print()

- **파일 실행 및 터미널 출력 2**

실행 – 디버깅 없이 실행(Ctrl + F5) – python Debugger 실행



변수(Variable)

- **챗GPT 프롬프트 입력**

+ 파이썬의 변수에 대해 설명해 줘

1️⃣ 변수란 무엇인가?

변수는 **값을 저장하는 공간에 붙이는 이름**입니다.

Python

```python
x = 10
```

- `x` → 변수 이름
- `=` → 할당 연산자
- `10` → 저장되는 값

👉 "x에 10을 저장한다" 라고 읽습니다.

2️⃣ 변수의 특징 (파이썬의 중요한 특성)

✔ **1. 자료형을 미리 선언하지 않는다**

파이썬은 **동적 타이핑(Dynamic Typing)** 언어입니다.

Python

```python
a = 10       # 정수(int)
a = "hello" # 문자열(str)
```

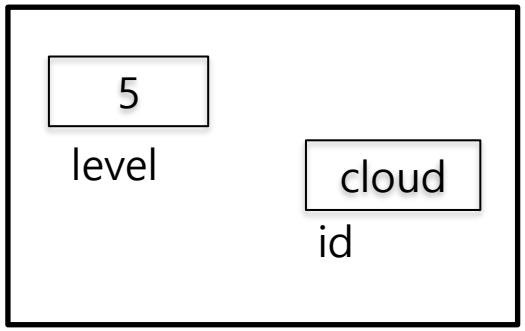
변수(Variable)

- **변수란?**
  - 프로그램에서 사용되는 자료를 저장하기 위한 메모리 공간(영역)
  - 프로그램 실행 중에 값 변경 가능, variable 이라 함

- **변수의 선언 및 초기화**
  - 변수 선언은 어떤 타입의 데이터를 저장할 것인지 그리고 변수이름은 무엇인지를 결정한다. (자료형)은 생략함.

    (자료형) **변수이름 = 초기값;**

    예) level = 5 , id = "cloud"



변수(Variable)

- **변수 사용 예제**

```
# 상수
print(4 + 5)  # 9

# 변수 선언과 초기화
n1 = 4
n2 = 5

print(n1 + n2)  # 9
```

```
n1 = 10  # n1의 값을 10으로 변경
print(n1 + n2)  # 15

# 문자 변수 선언
name = "선화"
age = 20

# 변수 출력
print("이름:", name)
print("나이:", age)
```

자료형(Data Type)

- **자료형이란?**
  - 사용할 데이터의 종류에 따라 메모리 공간을 적절하게 설정해 주는 것
  - 파이썬에서는 표기하지 않음 (예, num=10, name="홍길동")

분류	자료형	설명	예
정수	int	소수점이 없는 수	-2, -1, 0, 1, 2
실수	float	소수점(.)이 있는 수, 부동소수점수라고도 불린다.	-3.5, 0.0, 1.25
문자열	str	알파벳과 다른 문자로 이루어진 문장	"a", 'hello', "비"
논리형	bool	참과 거짓을 표현	True, False(2가지 값만 있음)

자료형(Data Type)

- **자료형(Data Type) 예제**

```
# 숫자형
n1 = 4
rate_of_birth = 0.81

print(type(n1))  # <class 'int'>
print(type(rate_of_birth))  # <class 'float'>

# 문자형
name = "영우"
print(type(name))  # <class 'str'>

# 불린형
is_raining = True
print(type(is_raining))  # <class 'bool'>
```

연산자(Operator)

▪ 연산자

파이썬의 연산자에 대해 알려주세요

+ 도구 사고 모델

2. 비교 연산자 (Comparison Operators)

두 값을 비교하여 참(`True`) 또는 거짓(`False`)을 반환합니다. 조건문(if)에서 정말 많이 쓰여요.

- `==` : 같다 (주의! `=` 는 할당, `==` 가 비교입니다.)
- `!=` : 같지 않다
- `>` / `<` : 크다 / 작다
- `>=` / `<=` : 크거나 같다 / 작거나 같다

---

3. 논리 연산자 (Logical Operators)

여러 조건을 조합할 때 사용합니다.

- `and` : 두 조건이 **모두** 참일 때만 `True`
- `or` : 두 조건 중 **하나라도** 참이면 `True`
- `not` : 논리값을 반대(뒤집기)로 바꿈 (`True` -> `False`)

---

4. 할당(대입) 연산자 (Assignment Operators)

변수에 값을 저장하거나, 연산과 동시에 값을 업데이트할 때 사용합니다.

연산자(Operator)

- **산술 연산자**

연산자	연산 작업	설명
+	n1 + n2	더하기
-	n1 - n2	빼기
*	n1 * n2	곱하기
/	n1 / n2	나누기
//	n1 // n2	몫
%	n1 % n2	나머지
**	n1 ** n2	거듭제곱

연산자(Operator)

- **산술 연산자 예제**

```
n1 = 10 # '='는 대입 연산자
n2 = 4

print("n1 + n2 =", n1 + n2)  # 덧셈
print("n1 - n2 =", n1 - n2)  # 뺄셈
print("n1 * n2 =", n1 * n2)  # 곱셈
print("n1 / n2 =", n1 / n2)  # 나눗셈
print("n1 // n2 =", n1 // n2)  # 몫
print("n1 % n2 =", n1 % n2)  # 나머지
print("n1 ** n2 =", n1 ** n2)  # 거듭제곱
```

```
n1 + n2 = 14
n1 - n2 = 6
n1 * n2 = 40
n1 / n2 = 2.5
n1 // n2 = 2
n1 % n2 = 2
n1 ** n2 = 10000
```

연산자(Operator)

몫과 나머지 계산하기

30개의 빵을 4명이 나눠 가질때 몫과 나머지를 구하세요.

☞ 실행 결과

```
몫: 7, 남은 빵: 2
```

연산자(Operator)

▪ 비교 연산자

연산자	의미	예	결과
<	보다 작다	9 < 10	True
>	보다 크다	9 > 10	False
<=	작거나 같다	9<=10	True
>=	크거나 같다	9>=10	False
==	같다	9==10	False
!=	같지 않다	9!=10	True

▪ 논리 연산자

연산자	규칙
x and y	x, y 가 모두 참이면 참, 나머지는 거짓
x or y	x, y 중 둘 중 하나가 참이면 참
not x	x가 참이면 거짓, 거짓이면 참

연산자(Operator)

- **비교 연산자 예제**

```
# 비교 연산자
x = 10
y = -10

print("x > y:", x > y)  # true
print("x < y:", x < y)  # false

print("x == y:", x == y)  # false
print("x != y:", x != y)  # true

# 논리 연산자
print("x > 0 and y > 0:", x > 0 and y > 0)  # false
print("x > 0 or y > 0:", x > 0 or y > 0)  # true
print("not (x > 0):", not (x > 0))  # false
```

입력 – input()

➢ input() – 입력 함수

```
"""
print("문자 입력: ")
ch = input()  # 사용자로부터 문자 입력 받기
print("입력한 문자:", ch)
"""

# 입력 안내 메시지와 함께 문자 입력 받기
ch = input("문자 입력: ")
print("입력한 문자:", ch)
```

```
문자 입력: banana
입력한 문자: banana
```

입력 – input()

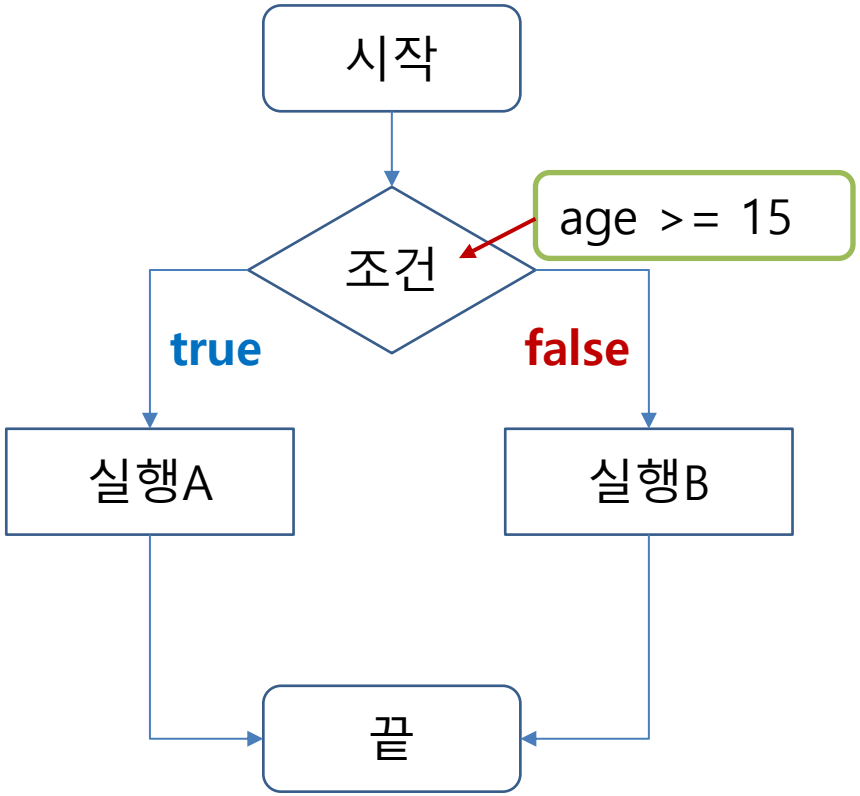
➢ input() – 입력 함수

```
# 숫자 입력 받기
# 입력은 숫자도 항상 문자열로 처리됨
num = input("숫자 입력: ")
print("입력한 숫자(문자열):", num)
print(type(num))  # <class 'str'>

num = int(num)  # 문자열을 정수로 변환
print("입력한 숫자(정수):", num)
print(type(num))  # <class 'int'>

print(num + 10)  # 문자열과 숫자 연산은 오류 발생

# 실수 입력 받기 - 몸무게의 2배 계산하기
weight = input("몸무게 입력(kg): ")
weight = float(weight)  # 문자열을 실수로 변환
print(type(weight))  # <class 'float'>
print("몸무게의 2배:", weight * 2)
```

조건문(Choice Structure)

조건문

- 특정한 조건에 의해서 프로그램 진행이 분기되는 구문



조건문(Choice Structure)

- **챗GPT 프롬프트 입력**

+ if문 사용하는 방법을 알려줘

1 기본 구조

Python

```python
if 조건식:
    실행할 코드
```

👉 **콜론(:)** 과 **들여쓰기(Indentation)** 가 매우 중요합니다.

2 간단한 예제

Python

```python
age = 20

if age >= 18:
    print("성인입니다.")
```

✔ `age >= 18` 이 True이면 실행
✔ False이면 아무것도 실행되지 않음

조건문(Choice Structure)

- **if 문**

```
if 조건식:
    수행문
```

4칸 들여쓰기
인덴트(indent)

조건식이 참이면 수행문 실행

- **if ~ else 구문**

```
• if 조건식:
      수행문 1
• else :
      수행문 2
```

조건식이 참이면 수행문1 실행,
  아니면 수행문2 실행

조건문(Choice Structure)

- if 문

```
age = 17
if age < 20:
    print("미성년자입니다.")

print(f"나이는 {age}세입니다.")
```

- if ~ else 구문

```
age = 17
if age < 20:
    print("미성년자입니다.")
else:
    print("성인입니다.")

print(f"나이는 {age}세입니다.")
```

조건문(Choice Structure)

- 비교문이 2개 이상인 경우

```
'''
조건
 컴활 필기점수는 60점 이상이고,
 실기점수는 70점 이상이면 합격
'''

필기 = int(input("필기 점수 입력(0~100): "))
실기 = int(input("실기 점수 입력(0~100): "))

if 필기 >= 60 and 실기 >= 70:
    print("합격입니다.")
else:
    print("불합격입니다.")
```

조건문(Choice Structure)

- **if ~ elif ~ else구문**

```
• if 조건1:
      수행문1
• elif 조건2:
      수행문2
• else :
      수행문 3
```

조건1이 참이면 수행문1 실행, 조건2가 참이면 수행문2 실행, 조건1,2가 모두 거짓이면 수행문3 실행

조건문(Choice Structure)

- 미니 프로그램 – 신호등

```
color = input("신호등 색상 입력(빨강/노랑/초록): ")

if color == "빨강":
    print("정지")
elif color == "노랑":
    print("주의")
elif color == "초록":
    print("진행")
else:
    print("잘못된 입력입니다")
```

반복문 – while문

- **while문**

4칸 들여쓰기
인덴트(indent)

```
초기값
while 조건문:
    실행문1
    증감값
```

반복문 – while문

- **while문 예제**

```
a = 1
print(a)

a += 1
print(a)

a += 1
print(a)
```

```
# 1부터 5까지 출력하는 프로그램
n = 1
while n <= 5:
    print(n)
    n += 1
print("반복문 종료")

# 5부터 1까지 출력하는 프로그램
n = 5
while n >= 1:
    print(n)
    n -= 1
print("반복문 종료")
```

반복문 – while문

- **반복 조건문**

  반복문에서 break 문을 만나면 더 이상 반복을 수행하지 않고, 반복문을 빠져 나옴

```
while True:
    수행문
    if 조건 :
        break
```

반복문 – while문

- **반복 조건문**

```
# 1부터 5까지 출력하는 프로그램 (break 사용)
n = 1
while True:
    if n > 5:
        break
    print(n)
    n += 1
print("반복문 종료")
```

반복문 – while

- **in 명령어**

```
# 문자열 내에 특정 문자나 문자열이 포함되어 있는지 확인
# in, not in
animals = "dog cat bird fish"
print("dog" in animals)  # True
print("lion" in animals)  # False
print("cat" not in animals)  # False
print("tiger" not in animals)  # True
```

반복문 – while

- **챗봇 프로그램**

```
from datetime import datetime

while True:
    user_input = input("챗봇에게 질문하세요 (종료하려면 'exit' 입력): ")
    if user_input == "exit":
        print("챗봇을 종료합니다.")
        break
    elif "안녕" in user_input:
        print("챗봇: 안녕하세요! 무엇을 도와드릴까요?")
    elif "이름" in user_input:
        print("챗봇: 제 이름은 챗봇입니다.")
    elif "시간" in user_input:
        now = datetime.now()
        print(f"챗봇: 현재 시간은 {now.hour}시 {now.minute}분입니다.")
    else:
        print("챗봇: 죄송합니다, 잘 모르겠어요.")
```

반복문 – for문

- **for문**

순서열의 각 원소를 처음부터 순회하면서 반복변수에 담아 낸다.

순서열은 리스트, 튜플, 문자열 등을 사용

for - in range() 와 for - in 문을 사용함.

```
for 반복변수 in range(시작값, 종료값, 증감값):
    실행문
```

```
for 반복변수 in 순서열:
    실행문
```

반복문 – for문

- range() 함수 사용하기

range(시작값, 종료값, 증감값):

✓ 시작값을 생략하면 **0**부터 시작하고, 종료값은 (**종료값-1**) 이다.

```
print(range(1, 6))  # range 객체 출력
print(list(range(1, 6)))  # 리스트로 변환하여 출력

print(range(5))  # range 객체 출력
print(list(range(5)))  # 리스트로 변환하여 출력

# 1부터 5까지 출력하는 프로그램
for n in range(1, 6):
    print(n)
print("반복문 종료")
```

반복문 – for문

- **for 반복문 예제**

```
# 1부터 10까지의 짝수 출력하는 프로그램
for n in range(1, 11):
    if n % 2 == 0:
        print(n)
print("반복문 종료")

# 1부터 10까지의 짝수 출력하는 프로그램 (step 이용)
for n in range(2, 11, 2):
    print(n)
print("반복문 종료")

# 1부터 10까지의 합계를 구하는 프로그램
total = 0
for n in range(1, 11):
    total += n
    print(f"n={n}, total={total}")
print(f"1부터 10까지의 합계: {total}")
```

중첩 for문

- **중첩 for문**

  - 행, 열 구현하기

```
for i in range(1, 6)
    for j in range(1, 6)
        실행문;
    print()
```

행

열

5행 5열

	열1	열2	열3	열4	열5
행1					
행2					
행3					
행4					
행5					

중첩 for문

- **중첩 for문**

```
가가가가
가가가가
가가가가
```

```python
for i in range(3):  # 바깥쪽 for문
    for j in range(4):  # 안쪽 for문
        print("가", end="")
    print()  # 줄바꿈
```

중첩 for문 – 구구단 전체

- **구구단 전체 출력 프로그램**

```
for dan in range(2, 10):  # 2단부터 9단까지
    for n in range(1, 10):  # 각 단의 1부터 9까지
        result = dan * n
        print(f"{dan} x {n} = {result}")
    print()  # 단이 바뀔 때마다 줄바꿈
```

```
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27

4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45

6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
```

리스트(List)

- **챗GPT 프롬프트 입력**

+ 파이썬 리스트 자료구조에 대해 설명해줘

1. 리스트란?

리스트는 여러 개의 값을 순서대로 저장하는 자료구조입니다.
대괄호 `[]` 를 사용해서 만들어요.

python 코드 복사

```python
numbers = [1, 2, 3, 4]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True]
```

👉 서로 **다른 자료형을 섞어서** 저장할 수 있다는 게 특징이에요.

리스트(List)

- **정수형 리스트**

```
# 리스트 생성
my_list = [1, 2, 3, 4, 5]

# 리스트 출력
print("리스트:", my_list)
print(type(my_list))

# 리스트의 길이 출력
print("리스트의 길이:", len(my_list))

# 리스트의 특정 요소 접근
print("리스트의 첫 번째 요소:", my_list[0])
print("리스트의 세 번째 요소:", my_list[2])
print("리스트의 마지막 요소:", my_list[-1])

# 리스트의 슬라이싱(':' 연산자 사용-부분 리스트 추출)
print("리스트의 첫 세 요소:", my_list[0:3])
print("리스트의 마지막 두 요소:", my_list[-2:])
print("리스트의 모든 요소:", my_list[:])
```

리스트(List)

- **정수형 리스트**

```
# 리스트 요소 변경
my_list[1] = 20
print("요소 변경 후 리스트:", my_list)

# 리스트에서 요소 제거 - remove() 메서드 사용
del my_list[2]
print("요소 제거 후 리스트:", my_list)

# 리스트에 요소 추가 - append() 메서드 사용
# my_list.append(6)
# print("요소 추가 후 리스트:", my_list)
```

리스트(List)

- for 변수 in 리스트

```
my_list = [10, 20, 30, 40, 50]
print(20 in my_list)  # True
print(25 in my_list)  # False

print("for문을 사용한 리스트 출력:")

for item in my_list:
    print(item, end=' ')
print()

# 40보다 큰 요소만 출력
print("40보다 큰 요소 출력:")

for item in my_list:
    if item > 40:
        print(item, end=' ')
print()
```

리스트(List)

- 리스트의 주요 메서드(함수)

```
# 요소 추가
fruits = ['사과', '바나나', '귤']
fruits.append('포도')  # 맨 끝에 요소 추가
print("append 후:", fruits)

# 요소 삽입
fruits.insert(1, '딸기')  # 인덱스 1에 요소 삽입
print("insert 후:", fruits)

# 요소 제거
fruits.remove('바나나')  # 값으로 요소 제거
print("remove 후:", fruits)

# 맨 마지막 요소 제거
last_fruit = fruits.pop()  # 맨 마지막 요소 제거 및 반환
print("pop 후:", fruits)
```

리스트(List)

- **if 변수 in [list]**

  리스트 내부에 값이 있으면 True, 없으면 False

```
# 음식 분류하기 - 한식, 일식, 중식
foods = ["비빔밥", "짜장면", "초밥", "김치찌게"]

for food in foods:
    if food in ["짜장면",  "짬뽕"]:
        print(f'{food}는(은) 중식입니다.')
    elif food in ["초밥", "우동"]:
        print(f'{food}는(은) 일식입니다.')
    else:
        print(f'{food}는(은) 한식입니다.')
```

```
비빔밥는(은) 한식입니다.
짜장면는(은) 중식입니다.
초밥는(은) 일식입니다.
김치찌게는(은) 한식입니다.
```

딕셔너리(Dicitionary)

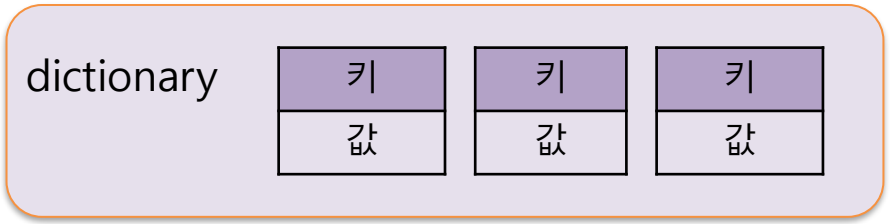
◆ **딕셔너리**

리스트 처럼 여러 개의 값을 저장할 수 있고, 키(key)와 값(value)으로 대응시켜 저장하는 자료구조이다.

중괄호{ }를 사용한다.

딕셔너리 이름 = **{ 키:값, 키:값....}**

{ 'name': '한국민', 'age': 28 }



딕셔너리(Dicitionary)

◆ 딕셔너리 주요 메서드

함수	사용 예
d[key] = value	d = {'Tomas':13, 'Jane':9} d['Mike'] = 10 **# 요소 추가** {'Tomas':13, 'Jane':9, 'Mike':10 }
del d[key]	del d['Jane'] **#요소 삭제** {'Tomas':13, 'Mike':10 }
d.pop(key)	d.pop('Mike') 10 {'Tomas':13}
clear()	d.clear() # d={ } 빈 딕셔너리
d.keys()	d.keys() **# 모든 키 가져오기** d_keys(['Tomas', 'Mike'])
d.values()	d.Values() **# 모든 값 가져오기** d_values([13, 10])
d.get(key)	d.get('Jane') #9

딕셔너리(Dicitionary)

◆ 딕셔너리 생성 및 관리

```
student = {
    "name": "한강",
    "age": 21,
    "major": "컴퓨터 공학"
}

print("딕셔너리 내용:", student)
print(type(student))  # <class 'dict'>

print("딕셔너리의 크기:", len(student))  # 3

print("딕셔너리의 요소 접근:")
print("이름:", student["name"])  # 한강
print("나이:", student["age"])    # 21
print("전공:", student["major"])  # 컴퓨터 공학

# 딕셔너리 요소 추가
student["university"] = "한강대학교"
print("요소 추가 후 딕셔너리:", student)
```

딕셔너리(Dicitionary)

```
# 딕셔너리 요소 수정
student["age"] = 22
print("요소 수정 후 딕셔너리:", student)

# 딕셔너리 요소 삭제
# del student["major"]
student.pop("major")  # pop 메서드를 사용하여 요소 삭제
print("요소 삭제 후 딕셔너리:", student)


# 키 목록과 값 목록
keys = student.keys()
values = student.values()
print("키 목록:", list(keys))
print("값 목록:", list(values))

print("딕셔너리 순회:")
for key, value in student.items():
    print(f"{key}: {value}")
```

딕셔너리(Dicitionary)

- **용어 사전 만들기**

```
♠ 컴퓨터 용어 사전 ♠
검색할 용어를 입력하세요 (종료: q or Q): 이진수
이진수: 0과 1로 이루어진 수 체계
검색할 용어를 입력하세요 (종료: q or Q): 함수
'함수'에 대한 정의를 찾을 수 없습니다.
검색할 용어를 입력하세요 (종료: q or Q): q
프로그램을 종료합니다.
```

1. Dictionary 자료구조에 컴퓨터 용어와 정의를 저장한다.
2. 용어를 계속 반복해서 검색 할 수 있다.
3. 검색한 용어가 없으면 정의된 단어가 없음을 알려준다.
4. 검색을 종료하려면 ‘q’ 또는 ‘Q’를 입력한다.

딕셔너리(Dicitionary)

- 용어 사전 만들기

```
print("♠ 컴퓨터 용어 사전 ♠")

dic = {
    "CPU": "Central Processing Unit (중앙 처리 장치)",
    "RAM": "Random Access Memory (임의 접근 메모리)",
    "이진수": "0과 1로 이루어진 수 체계",
    "알고리즘": "문제를 해결하기 위한 단계적 절차"
}
```

딕셔너리(Dicitionary)

- 용어 사전 만들기

```
while True:
    word = input("검색할 용어를 입력하세요 (종료: q or Q): ")

    if word.lower() == "q":
        print("프로그램을 종료합니다.")
        break

    definition = dic.get(word) # 입력한 단어에 대한 정의를 사전에서 가져옴

    if definition:
        print(f"{word}: {definition}")
    else:
        print(f"'{word}'에 대한 정의를 찾을 수 없습니다.")
```

튜플(tuple)

- **튜플(tuple)**
  - 튜플의 요소를 변경(추가, 수정, 삭제)할 수 없다.
  - 요소 추가는 초기화나 튜플간 합치기를 하면 가능함
  - 리스트처럼 동일한 방식으로 인덱싱과 슬라이싱 가능함
  - 소괄호( ) 를 사용한다.

튜플 이름 = **(요소1, 요소2....)**

```
t1 = ()
t2 = (1, )
t3 = (1, 2, 3)
t4 = ('a', 'b', 'c')
```

튜플(tuple)

- **튜플 자료형**

```
# 튜플 생성
t = (1, 2, 3)
print(t)  # (1, 2, 3)
print(type(t))  # <class 'tuple'>

# 특정 요소 접근
print("첫 번째 요소:", t[0])  # 1
print("두 번째 요소:", t[1])  # 2
print("마지막 요소:", t[-1])  # 3

# 튜플 길이
print("튜플의 길이:", len(t))  # 3

# 슬라이싱
print("첫 두 요소:", t[0:2])  # (1, 2)
print("두 번째 요소부터 끝까지:", t[1:])  # (2, 3)
```

튜플(tuple)

- **튜플 자료형**

```
# 튜플은 변경 불가능(immutable)
# t[0] = 10  # 오류 발생

# 튜플 삭제 불가능
# del t[1]  # 오류 발생

#  단일 요소 튜플 생성
t2 = (5)
print("단일 요소 튜플 아님:", t2)  # 5

t2 = (5,)
print("단일 요소 튜플:", t2)  # (5,)

# 튜플 연결
t3 = t + t2
print("튜플 연결:", t3)  # (1, 2, 3, 5)
```

튜플의 요소는 수정 및 삭제 할 수 없다.

문자열 인덱싱, 슬라이싱

- **문자열은 특별한 1차원 리스트이다.**

문자열(시작번호:끝번호)

※ 끝번호는 (끝번호 -1)과 같다

```
# 문자를 저장한 리스트
my_list = ['h', 'e', 'l', 'l', 'o']
print(my_list)
print(my_list[0]) # h
print(my_list[-1]) # 0
print(my_list[0:-1]) # ['h', 'e', 'l', 'l']
print(len(my_list)) #5

# 문자열 - 특별한 1차원 리스트
greeting = "hello"
print(greeting[0]) # h
print(greeting[-1]) # 0
print(greeting[0:-1]) # hell
print(len(greeting)) #5
```

문자열 함수

- **문자열 함수(메서드) 정리**

메서드	설명
split()	s = ‘banana, grape, kiwi' s = fruit.split(',') [구분기호로 나누고 리스트로 만듬] s [‘banana', ' grape', ' kiwi']
replace()	s = 'Hello, World' s = s.replace('World', 'Korea') [문자를 변경함] 'Hello, Korea’
find()	s = "Hello" s.find('H') 0 s.find('k') -1 [문자열이 존재하는 위치 반환. 없으면 -1반환]
strip()	s = “ Hi, lee” s.strip() Hi, lee

문자열 함수

- **문자열 함수(메서드)**

```
# 대소문자 변환
print("소문자 변환:", greeting.lower()) #hello
print("대문자 변환:", greeting.upper()) #HELLO

# 문자열 변경 - replace(변경전 문자, 변경후 문자)
s1 = "Hello, World"
print(s1.replace('World', 'Korea')) #Hello, Korea
```

```
# split() -> 문자열을 구분기호로 분리하는 함수
csv = "apple,banana,strawberry"
fruits = csv.split(',') #문자열이 리스트형태로 변환됨
print(fruits) #['apple', 'banana', 'strawberry']
print(fruits[0]) #apple
print(fruits[-1]) #strawberry
```

실습 문제 – 문자열 처리

2025년 민생회복 지원금 신청 프로그램을 아래와 같이 구현하세요.

출생연도 끝자리	접종 요일
1 또는 6	월요일
2 또는 7	화요일
3 또는 8	수요일
4 또는 9	목요일
5 또는 0	금요일

☞ 실행 결과

```
출생연도 입력: 2023
신청일은 수요일입니다.
```

```
출생연도 입력: 2010
출생연도는 1900년부터 2006년 사이여야 합니다.
```

실습 문제 – 문자열 처리

- 민생회복 지원금 신청

```
birth_year = input("출생년도 4자리를 입력하세요: ")
last_digit = birth_year[-1] #출생년도 끝자리 숫자
year = int(birth_year)
if year < 1900 or year > 2006:
    print("출생년도는 1900년부터 2006년 사이로 입력해주세요.")
else:
  if last_digit in ['1', '6']:
    print("신청일은 월요일입니다.")
  elif last_digit in ['2', '7']:
    print("신청일은 화요일입니다.")
  elif last_digit in ['3', '8']:
    print("신청일은 수요일입니다.")
  elif last_digit in ['4', '9']:
    print("신청일은 목요일입니다.")
  elif last_digit in ['5', '0']:
    print("신청일은 금요일입니다.")
```

2차원 리스트(list)

- **2차원 리스트의 선언 및 생성**
  - 리스트 내부에 리스트를 가진 자료 구조이다.
  - 행과 열의 표(테이블) 형태를 이루고 있다.

리스트 이름 = [요소1, 요소2, **[요소1, 요소2, 요소3]**]

	열1	열2
행1	a[0][0]	a[0][1]
행2	a[1][0]	a[1][1]
행3	a[2][0]	a[2][1]

2차원 리스트(list)

- **2차원 리스트 생성 및 출력**

```
matrix = [
    [1, 2, 3],  #1행
    [4, 5, 6]   #2행
]

# 출력
print(matrix) #[[1, 2, 3], [4, 5, 6]]
print(type(matrix)) #<class 'list'>

# 특정 행 출력
print(matrix[0]) #[1, 2, 3]
print(matrix[1]) #[4, 5, 6]

# 특정 값 출력
print(matrix[0][0]) # 0행 0열, 1
print(matrix[0][1]) # 0행 1열, 2
print(matrix[1][2]) # 1행 2열, 6
```

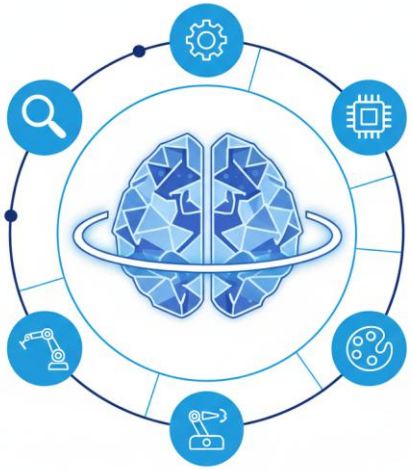
2차원 리스트(list)

- **2차원 리스트 생성 및 출력**

```
# 전체 값 출력
for row in matrix:  #행
    for val in row: #열 - 0행 [1, 2, 3], 1행 [4, 5, 6]
        print(val, end=" ") #1 2 3 4 5 6
    print() #행이 끝날 때 줄바꿈

# 요소 변경
matrix[0][1] = 20

# 요소 추가
matrix.append([7, 8, 9]) #3행에 [7, 8, 9] 추가
print(matrix) #[[1, 20, 3], [4, 5, 6], [7, 8, 9]]

# 요소 삭제
del matrix[1] #2행 삭제
print(matrix) #[[1, 20, 3], [7, 8, 9]]
```

파이썬 객체와 바이브 코딩

함수/ 클래스/ 모듈



목 차

- 파이썬 객체와 바이브 코딩
  - 함수
  - 클래스
  - 바이브 코딩(깃허브 코파일럿)
  - 모듈
  - 윈도우 프로그래밍

함수(function)

+ 파이썬의 함수에 대해 알려줘

1️⃣ 함수란?

함수는 **특정 작업을 수행하는 코드 묶음**이에요.

👉 한 번 만들어 두면

- 여러 번 사용할 수 있고
- 코드가 깔끔해지고
- 수정도 쉬워져요

예: "두 수를 더하는 기능", "이름을 출력하는 기능" 같은 것들

함수(function)

- **사용자 정의 함수의 형태**

```
def 함수 이름():
    함수의 내용
```

4칸 들여쓰기
인덴트(indent)

return 반환값이 없는 함수

```
def 함수 이름(매개 변수):
    함수의 내용
    return 반환값
```

return 반환값이 있는 함수

함수(function)

- **return이 없는 함수**

```
def greet():
    print("Hello, World!")

def greet2(name):
    print(f"Hello, {name}!")

greet()  # Hello, World!
greet2("Elsa")  # Hello, Elsa!
greet2("명제")  # Hello, 명제!
```

함수의 정의

함수의 호출

함수(function)

- **return이 있는 함수**

```
def message():
    return "Good Luck!"

def square(x):
    return x * x

def add(a, b):
    return a + b

print("5의 제곱:", square(5))  # 5의 제곱: 25
result = add(3, 5)
print("3 + 5 =", result)  # 3 + 5 = 8
```

함수(function)

절대값을 계산하는 함수를 구현하세요.

(10을 입력하면 10이 출력되고, -10을 입력해도 10이 출력됨)

☞ 실행 결과

```
절댓값: 10
```

함수(function)

- **변수의 유효 범위 – 전역 변수**

  전역 변수는 메인 함수 영역에 선언하여 사용하고, 영향 범위가 전체로 미친다. 프로그램이 종료되면 메모리에서 소멸한다.

```
def get_price():
    price = 1000 * quantity #price - 지역 변수
    print(f"{quantity}개에 {price}원 입니다.")

# 전역 변수
quantity = 2  # 수량
get_price()   # 호출(사용)

# 변수 출력
# print(price) # 소멸된 변수임(오류 발생)
```

함수(function)

- **변수의 유효 범위 – 지역 변수**

  지역변수는 함수나 명령문(조건, 반복)의 블록 안에서 생성되며 블록{ }을 벗어나면 메모리에서 소멸한다.

```
def click_a():
    x = 0  #지역변수
    x = x + 1
    print("x =", x)

click_a() # x = 1
click_a() # x = 1
click_a() # x = 1
print("==========")
```

함수(function)

- **변수의 유효 범위 – 정적 변수**

  지역변수가 정적 변수화 된 것으로 프로그램이 종료되면 메모리에서 소멸한다. **global** 키우드를 붙인다.

```
def click_b():
    global x  #지역변수가 전역변수화 함
    x += 1
    print("x =", x)

x = 0  #전역변수
click_b()  #x = 1
click_b()  #x = 2
click_b()  #x = 3
```

함수(function)

❖ 내장 함수(Built in Function)

함수	설명	사용 예
sum(iterable)	리스트나 튜플의 모든 요소의 합을 반환	sum([1, 2, 3]) 6 sum((1.2.3)) 6
max(iterable)	리스트나 튜플의 최대값을 반환	max([1, 2, 3]) 3 max((1.2.3)) 3
round(n, digit)	숫자를 입력받아 반올림하여 돌려줌	round(4.6) 5 round(4.4) 4
eval(expression)	문자열 표현식을 숫자로 변환	eval('1+2') 3

함수(function)

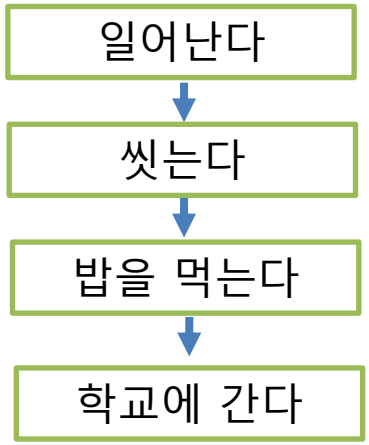
❖ 내장 함수(Built in Function)

```
a = [1, 2, 3, 4]
b = (1, 2, 3, 4)

print(sum(a))  # 합계
print(sum(b))

print(max(a))  # 최대값
print(max(b))

print(min(a))  # 최소값
print(min(b))
```

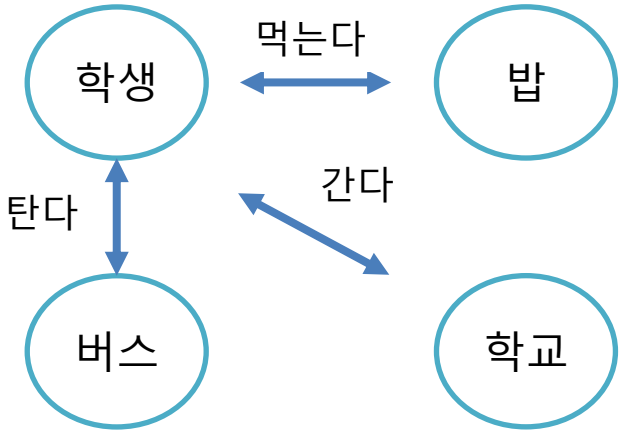
```
# 반올림
print(round(2.74))  #3
print(round(2.14))  #2

# 소수 자리수
x = 706.351
print(round(x, 1)) # 소수첫째자리, 706.4
print(round(x, 0)) # 706.0
print(round(x)) # 정수 706
print(int(round(x, -1))) # 일의 자리 710
```

클래스와 객체

- **객체(Object)란?**
  - "의사나 행위가 미치는 대상" -> 사전적 의미
  - 구체적, 추상적 데이터 단위 (구체적- 책상, 추상적-회사)
- **객체지향 프로그래밍(Objected Oriented Programming)**
  - 객체를 기반으로 하는 프로그래밍
  - 먼저 객체를 만들고 객체 사이에 일어나는 일을 구현함.



<절차지향 –C언어>



<객체지향 – Python>

클래스와 객체

- **클래스란?**
  - 객체 지향 프로그래밍(OOP)의 기본 개념 중 하나로, 관련된 속성과 메서드를 하나의 단위로 묶는 틀입니다.
  - 클래스를 사용하면 코드의 재사용성과 유지보수성을 높일 수 있습니다.
  - 클래스는 설계도와 같으며, 이를 기반으로 여러 객체(인스턴스)를 생성할 수 있습니다.
- **클래스의 속성과 기능**
  - 객체의 특성(property), 속성(attribute) -> **멤버 변수**
  - 객체가 하는 기능 -> **멤버 함수**

클래스와 객체

- 클래스 정의 및 사용

```
class 클래스 이름 :
    def __init__(self):
        멤버변수

    def 함수이름(self):
        return
```

```
객체(인스턴스) = 클래스( )

객체.멤버변수
객체.멤버 메서드
```

객체를 생성한 후 점(.) 연산자를 사용하여 멤버변수나 메서드에 접근함

클래스와 객체

- **생성자(constructor)**
  - 객체가 생성될때 자동으로 호출되는 특별한 메서드
  - 생성자는 **__init__(매개변수)**의 형태로 작성하고, 리턴값이 없다
  - 클래스 내의 모든 함수(메서드)의 매개변수에 **self**를 넣어줌

Bike	
color gears	속성(Attributes)
ride()	메서드(Methord)

클래스와 객체

- **Bike 클래스 정의 및 사용**

```
# Bike 클래스 정의
class Bike:
    # 생성자 메서드
    def __init__(self, color , gears):
        self.color = color
        self.gears = gears

    # 문자열 표현 메서드
    def __str__(self):
        return f"Bike(자전거 색상={self.color}, 기어 수={self.gears})"

    def ride(self):
        print(f"{self.color} 자전거를 타고 있습니다.")

my_bike = Bike("black", 10)
print(my_bike)
my_bike.ride()
```

클래스와 객체

- **쇼핑몰 장바구니(Cart)**

```python
class Cart:
    # 초기화 메서드
    def __init__(self):
        self.items = []  # 아이템을 저장하는 리스트

    # 아이템 추가 메서드
    def add_item(self, item):
        self.items.append(item)

    # 아이템 제거 메서드
    def remove_item(self, item):
        if item in self.items:
            self.items.remove(item)

    # 아이템 조회 메서드
    def get_items(self):
        return f"장바구니: {self.items}"
```

클래스와 객체

- **쇼핑몰 장바구니(Cart)**

```
# 장바구니 객체 생성
cart = Cart()
cart.add_item("사과")
cart.add_item("바나나")
print(cart.get_items())  # 장바구니: ['사과', '바나나']

cart.remove_item("사과")
print(cart.get_items())  # 장바구니: ['바나나']
```

깃허브 코파일럿(copilot)

❖ 생성형 AI 코파일럿

▪ 챗GPT 프롬프트 입력

+ GitHub Copilot을 사용하는 방법을 알려줘

✔ **GitHub 계정**

- https://github.com ↗
- 로그인만 하면 OK

✔ **지원되는 에디터**

가장 많이 쓰는 건:

- **VS Code** ⭐ (추천)
- JetBrains (PyCharm, IntelliJ 등)
- Visual Studio

👉 여기서는 **VS Code 기준**으로 설명할게요.

✔ **GitHub Copilot 구독**

- 개인: 유료 (무료 체험 있음)
- 학생/교사: 무료
- 기업: 별도 플랜

깃허브 코파일럿(copilot)

❖ 깃허브 코파일럿

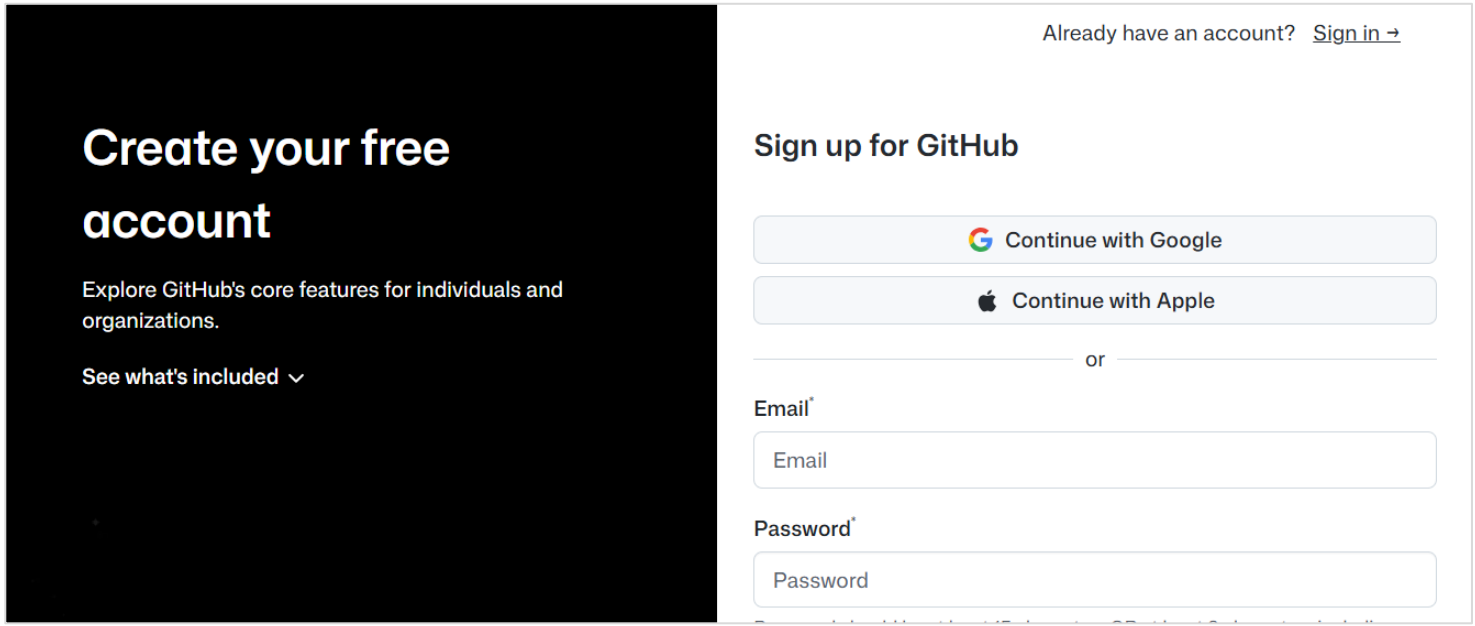
깃허브 코파일럿은 개발자가 코드를 더 빠르고 효율적으로 작성할 수 있도록 설계되었습니다. 프로그래머가 작성 중인 코드를 분석하고, 컨텍스트를 이해해 자동으로 코드를 제안합니다. 인터넷에 공개된 오픈 소스들로 학습했습니다.
깃허브(GitHub)가 출시한 프로그래밍을 돕는 인공지능입니다. 오픈 AI의 GPT 언어 모델을 이용해 깃허브 리포지터리를 학습시켜 만든 프로그램에 특화시켰습니다.

구독료 : 한 달에 10달러

깃허브 코파일럿(copilot)

❖ 깃허브 계정 만들기

https://github.com/ > 회원가입(sign up)

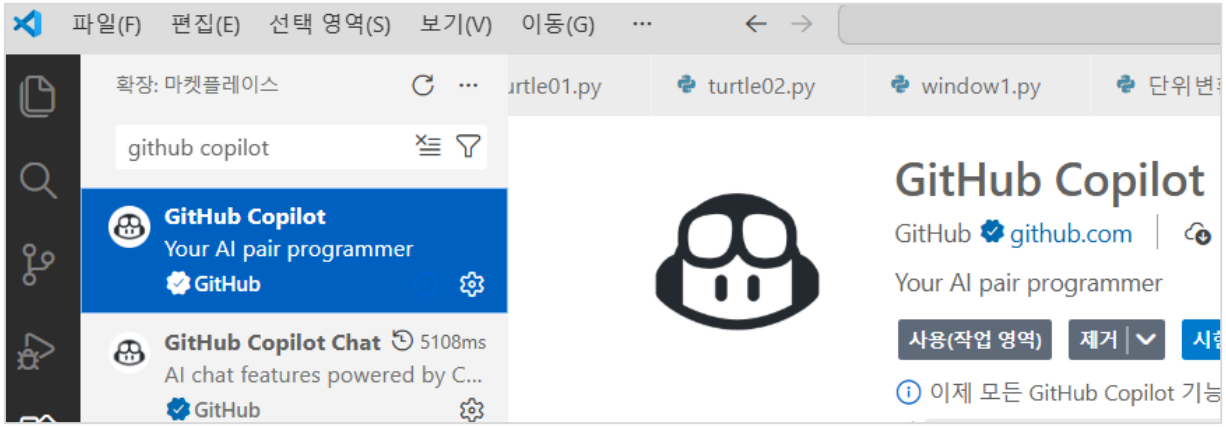


깃허브 코파일럿(copilot)

❖ **VS Code에서 GitHub Copilot 사용하기**

확장(Extension) > github copilot 검색 > 설치

< GitHub Copilot, GitHub Copilot Chat >



모듈(Module)과 패키지

- **모듈(module)**

  모듈은 Python 파일 하나를 의미하며, 다른 Python 프로그램에서 모듈의 함수, 클래스 등을 재사용할 수 있다.

- **주요 모듈**

모 듈	설 명
math	수학 계산과 관련된 모듈
datetime	날짜 및 시간과 관련된 모듈
time	시간과 관련된 모듈
random	난수를 발생시키는 모듈
os	운영 체제(OS) 자원 제어 관련 모듈

math 모듈

- **math 모듈**

```python
import math

# 올림
print(math.ceil(2.54))

# 반올림 - math 모듈이 아님
print(round(2.54))

# 내림(버림)
print(math.floor(2.54))

# 제곱근 - 실수로 반환
print(math.sqrt(2))
print(math.sqrt(25))
```

```
3
3
2
1.4142135623730951
5.0
```

math 모듈

- **calendar 모듈**

```
import calendar

# 2026년 전체 달력 출력
calendar.prcal(2026)

# 2026년 3월 달력 출력
calendar.prmonth(2026, 3)

# 요일 이름 출력
print("요일 이름:", calendar.day_name[0])  # 월요일
print("요일 이름:", calendar.day_name[6])  # 일요일

print("요일 리스트:", calendar.day_name[:])
print("요일 리스트:", list(calendar.day_name))

# 특정 날짜의 요일 변환
day_of_week = calendar.weekday(2026, 3, 28)
print("특강일 요일 번호:", day_of_week)  # 5
print("특강일의 요일 이름:", calendar.day_name[day_of_week])  # 토요일
```

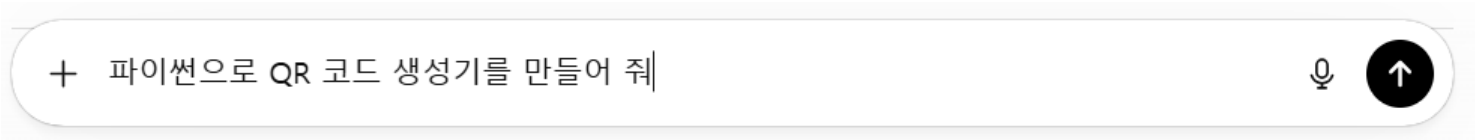
datetime 모듈

- **datetime 모듈**

```
# 날짜와 시간 다루기
from datetime import datetime, date

# 현재 날짜와 시간 출력
now = datetime.now()
print("현재 날짜와 시간:", now)

# 특정 날짜 생성
the_day = date(2026, 5, 5)
print("어린이날:", the_day)

# 오늘 날짜 출력
today = date.today()
print("오늘 날짜:", today)

# 날짜 차이 계산
date_diff = the_day - today
print("어린이날까지 남은 일수:", date_diff.days)
```

QR 코드 생성기

- **챗GPT 프롬프트 입력**



- qrcode 모듈 설치하기

  pip install qrcode

- qr 코드

  ☞ https://www.kaisa.or.kr/

QR 코드 생성기

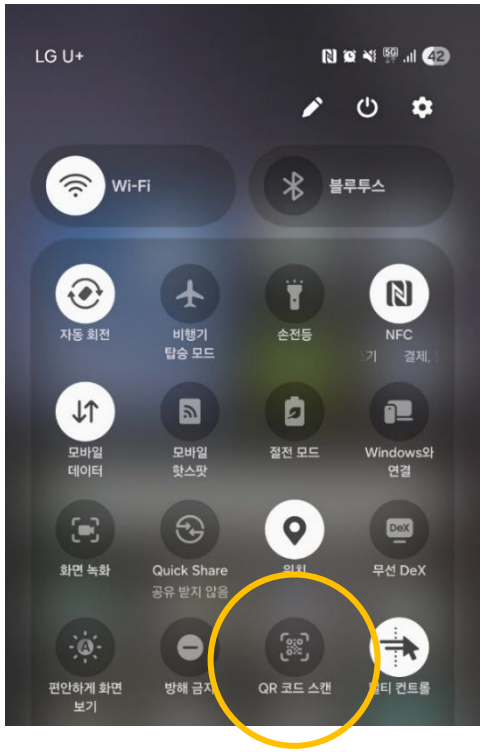
- **QR 코드 – qr.py**

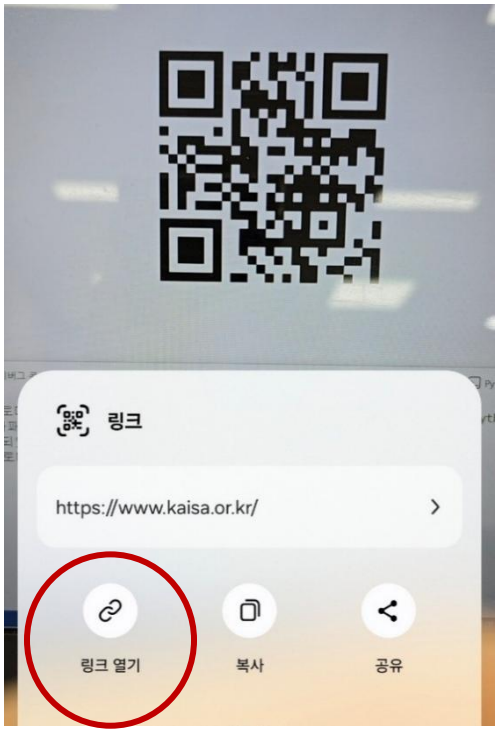
```python
import qrcode

# QR 코드에 담을 데이터
data = "https://www.kaisa.or.kr/"

# QR 코드 생성
img = qrcode.make(data)

# 이미지 저장
img.save("qrcode.png")

print("QR 코드가 생성되었습니다!")
```

QR 코드 생성기

- **스마트 폰으로 스캔**

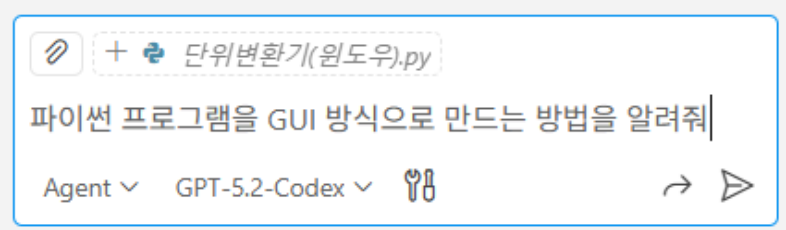
QR 코드 스캔 아이콘 선택 > 링크 열기 > 홈페이지로 이동





윈도우 프로그래밍

- **코파일럿 프롬프트 입력**



- **GUI 라이브러리 선택**: 표준 `tkinter`(가장 간단), `PyQt/PySide`(기능 풍부), `wxPython` 등
- **기본 흐름**: `창 생성 → 위젯 배치 → 이벤트 연결 → 실행`

예) `tkinter` 최소 구조

- `Tk()`로 창 생성
- `Label`, `Entry`, `Button` 등 위젯 추가
- `command`로 버튼 클릭 이벤트 연결
- `mainloop()`로 실행

윈도우 프로그래밍

- **GUI(Graphical User Interface)란?**

  그래픽 사용자 인터페이스를 줄여서 GUI라고 한다. GUI는 '화면'에 표시된 메뉴나 버튼으로 사용자와 상호 작용을 하는 간단한 프로그램이다.

  tkinter 라이브러리를 사용한다. -> **import tkinter**

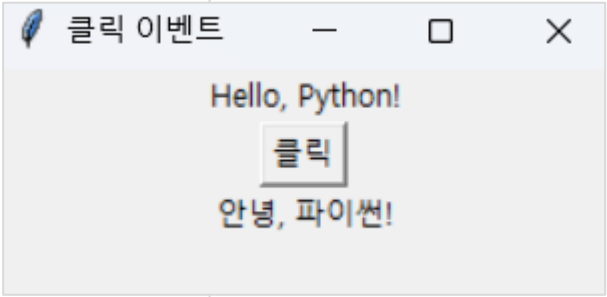
1. Tk 루트

개체이름	클래스
루트	Tk()
프레임	Frame
레이블	Label
입력상자	Entry
버튼	Button
출력상자	Text

윈도우 프로그래밍

- **클릭 이벤트 – 버튼을 눌렀을때 문자 출력**

```python
from tkinter import *

# 버튼 클릭시 호출되는 함수
def click():
    output.config(text="안녕, 파이썬!")

root = Tk() # 윈도우 창 생성
root.title("클릭 이벤트") # 창 제목 설정
root.geometry("250x100+200+100") # 가로x세로+x좌표+y좌표

# 라벨과 버튼 추가
# pack(): 위젯을 창에 추가하는 메서드
Label(root, text="Hello, Python!").pack()
Button(root, text="클릭", command=click).pack()

# 클릭후 출력 라벨
output = Label(root, text="")
output.pack()

# 이벤트 루프 시작
root.mainloop()
```



- **Button(버튼) – command**

Button(frame, text="확인", command=**click**).pack()

※ click에 괄호를 하면 함수 생성시점에서 작동하고, 괄호를 생략하면 클릭이 발생한 때 작동함

1. 콘솔에 출력

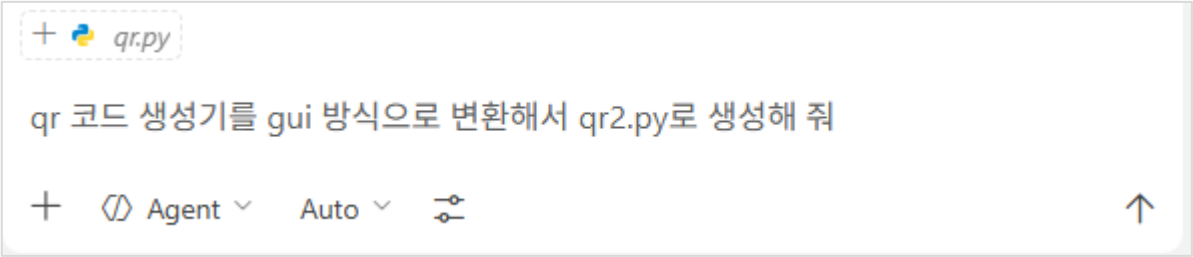
```
def click():
    print("안녕~ 파이썬!")
```

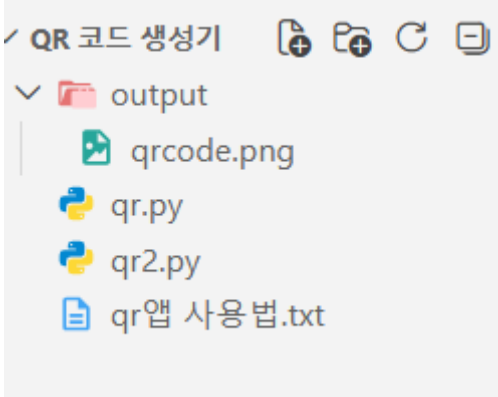
2. 레이블에 출력

```
def click():
    output.config(text="안녕, 파이썬!")
```

QR 코드 생성기

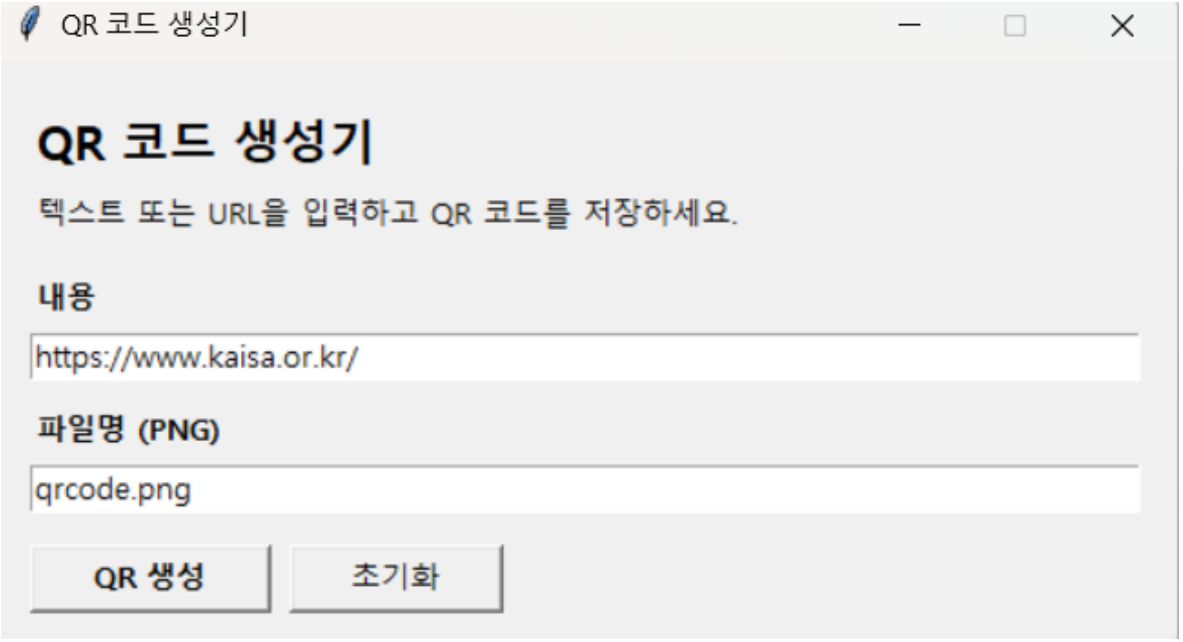
- **QR 코드 생성기**





QR 코드 생성기

- QR 코드 생성기



QR 코드 생성기

- **qr앱 사용법.txt**

qr코드 생성기 사용법.txt

사용자 입장에서 qr코드 생성기 작동법만 알려주는 가이드 형식으로 qr앱 사용법.txt로 만들어 줘

Agent Auto

```
[QR앱 사용법]

이 문서는 사용자 입장에서 QR코드 생성기 사용 방법만 안내합니다.

1. 앱 실행하기
1) 폴더에서 qr2.py를 실행합니다.
2) "QR 코드 생성기" 창이 열리면 준비 완료입니다.

2. QR코드 만들기
1) "내용" 칸에 QR로 만들 문구 또는 주소를 입력합니다.
   - 예: https://www.naver.com
2) "파일명(PNG)" 칸에 저장할 이름을 입력합니다.
   - 예: my_qr.png
3) "QR 생성" 버튼을 누릅니다.
4) 완료 팝업이 뜨면 생성이 끝난 것입니다.
```

QR 코드 생성기

- **QR 코드 생성기**

```
3. 저장 위치 확인
- 생성된 파일은 output 폴더에 저장됩니다.
- 예: output\my_qr.png

4. 초기화 방법
- "초기화" 버튼을 누르면 입력칸이 기본값으로 돌아갑니다.

5. 자주 있는 상황
- 파일명에 .png를 안 써도 자동으로 붙습니다.
- 파일명을 비우면 기본 이름(qrcode.png)으로 저장됩니다.
- 같은 파일명이 있으면 기존 파일이 바뀔 수 있으니 다른 이름을 권장합니다.
```